

Jobsheet 10 — Autentikasi & Manajemen Sesi

Sub-CPMK: Menerapkan autentikasi & manajemen sesi pengguna.

Perubahan dari Jobsheet 9

- Tambah `sql/02_users.sql` — tabel `users` (nama, username, password, role).
- Tambah `auth/register.php` + `proses_register.php` (password disimpan dengan `password_hash()`, cek username duplikat), `auth/login.php` + `proses_login.php` (`password_verify()`), `auth/logout.php` (`session_destroy()`).
- Tambah `includes/auth.php` — guard clause: redirect ke `auth/login.php` bila `$_SESSION['user_id']` belum ada. **Wajib di-include sebagai baris pertama** (sebelum `header.php`) agar `header('Location: ...')` masih bisa dipanggil sebelum ada output HTML.
- `includes/header.php`: `session_start()` diubah jadi `if (session_status() === PHP_SESSION_NONE)` agar tidak konflik dengan `auth.php` yang juga memulai session; navbar kini menampilkan nama petugas + Logout jika sudah login, atau link Login jika belum.
- Halaman yang **dikunci** (butuh login): `buku/tambah.php`, `buku/edit.php`, `buku/proses_tambah.php`, `buku/proses_edit.php`, `buku/hapus.php`, seluruh halaman `anggota/*`.
- Halaman yang **tetap publik**: `index.php` (Beranda) dan `buku/list.php` (katalog buku bisa dilihat Tamu tanpa login — sesuai wireframe Jobsheet 4).

Persiapan database

Jalankan skema tambahan:

```
psql -d simpus_mini -f sql/02_users.sql
```

Cara menjalankan

Opsi 1 — PHP built-in server:

```
php -S localhost:8000
```

Uji: akses `http://localhost:8000/buku/tambah.php` langsung tanpa login → harus redirect ke halaman Login. Daftar akun via Register, login, coba akses halaman yang sama → berhasil.

Opsi 2 — Laragon (Apache): lewat virtual host langsung ke folder `jobsheet-10/` (mis. `http://jobsheet10.test/`), atau bersarang di bawah domain proyek (mis. `http://dp2026.test/kode-praktikum/jobsheet-10/`) — path CSS/JS/link/redirect login sudah relatif otomatis (lihat `includes/header.php` & `includes/auth.php`), jadi keduanya jalan.

Catatan

- Guard `auth.php` sudah diverifikasi mengembalikan HTTP 302 ke `auth/login.php` untuk halaman terkunci meski database belum tersambung (guard berjalan sebelum kode butuh koneksi DB).
- Perbedaan akses berdasarkan `role` (mis. hanya `admin` boleh hapus anggota) belum diterapkan di jobsheet ini — jadi tugas mandiri.

Wireframe & User Flow — SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota — Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow — Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow — Pengembalian Buku

[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]
-> [Kembali ke Dashboard]

Wireframe: Halaman Login

```
+-----+
|          SIMPUS-Mini          |
+-----+
|          [ Login Petugas ]    |
|                               |
| Username : [_____]         |
| Password : [_____]         |
|                               |
|          [   Masuk   ]       |
|                               |
|  Belum punya akun? Daftar di sini  |
+-----+
```

Wireframe: Dashboard Petugas

```
+-----+
| SIMPUS-Mini   Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |
+-----+
| [Total Buku]   [Total Anggota]   [Sedang Dipinjam] |
|                                                       |
| Aksi Cepat:                                         |
| [ + Peminjaman Baru ]   [ + Pengembalian ]         |
|                                                       |
| Transaksi Terbaru                                   |
| -----|
| Anggota | Buku | Tgl Pinjam | Status                |
+-----+
```

Wireframe: Form Peminjaman

```

+-----+
|  Form Peminjaman Buku  |
|-----|
|  Anggota : [ dropdown pilih anggota ] |
|  Buku      : [ dropdown, hanya stok>0 ] |
|  Tanggal Pinjam : [ auto: hari ini ] |
|                                     |
|          [ Simpan Peminjaman ] |
+-----+

```

Wireframe: Form Pengembalian

```

+-----+
|  Pengembalian Buku  |
|-----|
|  Cari transaksi aktif: |
|  [ nama anggota / judul buku _____ ] |
|                                     |
|  Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+

```

Wireframe: Riwayat Peminjaman per Anggota

```

+-----+
|  Riwayat Peminjaman – Siti Aminah  |
|-----|
|  Buku          | Pinjam   | Kembali | Status      |
| Laskar Pelangi | 01/07   | 10/07   | Selesai     |
| Bumi Manusia  | 15/07   | -       | Dipinjam    |
+-----+

```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.

- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Dokumentasi Jobsheet 10 – Autentikasi & Manajemen Sesi

Dokumentasi ini melanjutkan [dokumentasi jobsheet-09](#) (CRUD Penuh). Jobsheet-10 mewujudkan sesuatu yang sudah dirancang **jauh** sebelumnya**: ingat wireframe halaman Login dan pembagian aktor** Tamu/Petugas yang sudah dibahas di [dokumentasi jobsheet-04](#) – sekarang, 6 jobsheet kemudian, fitur itu **benar-benar dibangun**.

Tentang docs/wireframe.md

File ini **identik persis** dengan [docs/wireframe.md](#) [di jobsheet-09](#).

Apa yang Baru di Jobsheet 10?

Sesuai [README.md](#) jobsheet ini:

1. `sql/02_users.sql` – tabel `users` baru (nama, username, password, role).
2. **Registrasi** (`auth/register.php` + `proses_register.php`) – password disimpan **terenkripsi** lewat `password_hash()`, dengan pengecekan username duplikat.
3. **Login** (`auth/login.php` + `proses_login.php`) – memverifikasi password lewat `password_verify()`.
4. **Logout** (`auth/logout.php`) – mengakhiri sesi lewat `session_destroy()`.
5. `includes/auth.php` – "penjaga gerbang" yang mengalihkan pengunjung yang belum login ke halaman Login, dipasang di semua halaman yang **wajib** login.
6. **Navbar dinamis** – menu dan status login/logout kini berubah tergantung apakah pengunjung sudah login atau belum.

Mengingat Kembali: Aktor Tamu vs Petugas

Ingat dari [dokumentasi jobsheet-04 §4](#), `wireframe.md` sejak awal membedakan 2 aktor:

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

Jobsheet ini **mewujudkan pembagian itu secara teknis:**

Halaman	Akses
index.php (Beranda)	Publik – Tamu boleh mengakses
buku/list.php (katalog buku)	Publik – Tamu boleh mengakses
buku/tambah.php, buku/edit.php, buku/hapus.php	Terkunci – wajib login
Seluruh halaman anggota/*	Terkunci – wajib login

Daftar Isi

1. [Konsep Dasar Autentikasi & Otorisasi](#)
2. [Tabel `users` & Registrasi](#)
3. [Login & Logout](#)
4. [Guard Halaman: `includes/auth.php`](#)
5. [Navbar Dinamis & CSS Pendukung](#)
6. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```

jobsheet-10/
├── index.php                # Tidak berubah – tetap publik
├── includes/
│   ├── auth.php            # BARU – guard clause login
│   ├── header.php          # Navbar & status login kini dinamis
│   ├── footer.php, koneksi.php # Tidak berubah
├── auth/
│   ├── register.php, proses_register.php # BARU
│   ├── login.php, proses_login.php      # BARU
│   └── logout.php                      # BARU
├── sql/
│   ├── 01_buku_anggota.sql
│   └── 02_users.sql              # BARU – tabel users

```

```

├─ buku/
│   └─ list.php                # Tidak berubah – tetap publik
│   └─ tambah.php, edit.php, hapus.php, proses_*.php # + require auth.php
├─ anggota/                    # SEMUA halaman + require auth.php
├─ docs/wireframe.md           # Identik dengan jobsheet-09
├─ README.md
└─ Dokumentasi/                # Folder dokumentasi ini

```

Catatan penting dari [README.md](#) jobsheet ini: `includes/auth.php` sudah diverifikasi tetap mengembalikan redirect ke halaman Login **meski database belum tersambung** – karena guard ini berjalan **sebelum** kode yang membutuhkan koneksi database. Selain itu, **perbedaan akses berdasarkan role** (misalnya hanya admin yang boleh menghapus anggota) **belum** diterapkan di jobsheet ini – dijadikan tugas mandiri (lihat [bab 6](#)).

1. Konsep Dasar Autentikasi & Otorisasi

1.1 Autentikasi vs Otorisasi: Dua Istilah yang Sering Tertukar

Istilah	Pertanyaan yang Dijawab	Contoh di Jobsheet Ini
Autentikasi (<i>authentication</i>)	"Kamu siapa ?"	Proses Login – memverifikasi username & password (bab 3)
Otorisasi (<i>authorization</i>)	"Kamu boleh melakukan apa?"	<code>includes/auth.php</code> – memeriksa apakah pengunjung boleh membuka halaman tertentu (bab 4)

Ingat istilah "otorisasi" sudah disinggung sejak [dokumentasi jobsheet-04 §4.4](#) – saat itu dijelaskan otorisasi **belum bisa** dibangun karena baru sebatas HTML/CSS statis. Sekarang, dengan PHP dan `$_SESSION` yang sudah kamu kuasai sejak jobsheet-07, keduanya **sama-sama** bisa diwujudkan: autentikasi lewat Login, otorisasi lewat guard clause di [bab 4](#).

1.2 Kenapa Password Tidak Boleh Disimpan Apa Adanya?

Bayangkan tabel `users` menyimpan password **persis seperti yang** diketik** pengguna (disebut *plaintext*). Kalau suatu saat database** ini bocor (diretas, di-backup ke tempat yang tidak aman, dll.), **semua** password pengguna langsung diketahui pihak yang tidak berhak — dan karena banyak orang memakai password yang sama di berbagai layanan, kebocoran ini bisa berdampak jauh melampaui aplikasi SIMPUS-Mini itu sendiri.

Hashing adalah solusinya: password diubah lewat fungsi matematika satu arah menjadi "acakan" karakter yang **tidak bisa dikembalikan** ke password aslinya. PHP punya fungsi bawaan untuk ini:

- `password_hash($password, PASSWORD_DEFAULT)` — mengubah password asli jadi hash, dipakai saat Registrasi ([bab 2](#)).
- `password_verify($password, $hash)` — memeriksa apakah sebuah password **cocok** dengan hash yang tersimpan, **tanpa** perlu mengetahui password aslinya — dipakai saat Login ([bab 3](#)).

Detail cara kerja kedua fungsi ini dibahas di bab-bab berikutnya.

1.3 Kolom `password` Bertipe `VARCHAR(255)` — Bukan Kebetulan

Ingat dari [dokumentasi jobsheet-08 §2.4](#), `VARCHAR(255)` membatasi panjang teks maksimal 255 karakter. Hasil dari `password_hash()` — dibahas di [bab 2](#) — selalu berupa teks acakan yang **panjang** (biasanya sekitar 60 karakter untuk algoritma default PHP saat ini, tapi bisa berbeda tergantung algoritma yang dipakai `PASSWORD_DEFAULT` di masa depan). `VARCHAR(255)` sengaja dipilih **cukup longgar** supaya muat menampung hash ini, **berapa pun** panjang persisnya.

1.4 Session: Mengingat "Siapa yang Login" Antar Halaman

Ingat konsep session dari [dokumentasi jobsheet-07 §3](#): `$_SESSION` menjembatani data antar permintaan HTTP yang terpisah. Di jobsheet-07, `$_SESSION` dipakai menyimpan **data buku/anggota** sementara. Di jobsheet ini, `$_SESSION` dipakai untuk tujuan yang **jauh** lebih umum** dalam pengembangan web: menyimpan **identitas pengguna yang sedang login** (`$_SESSION['user_id']`, `$_SESSION['nama']`,** `$_SESSION['role']` — dibahas di [bab 3](#)), supaya server "ingat" siapa yang sedang mengunjungi di **setiap** halaman berikutnya, tanpa pengguna perlu login ulang di setiap klik.

Dengan bekal konsep ini, kamu siap membaca implementasi sungguhnya mulai bab 2.

Lanjut ke: [Tabel `users` & Registrasi](#)

2. Tabel users & Registrasi

2.1 Skema sql/02_users.sql

```
-- Jobsheet 10: tabel users (Petugas) untuk autentikasi
-- Jalankan: psql -d simpus_mini -f sql/02_users.sql
```

```
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(255) NOT NULL,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'petugas'
);
```

Semua elemen di sini sudah kamu kenal dari [dokumentasi jobsheet-08 §2](#): SERIAL PRIMARY KEY untuk id yang bertambah otomatis, NOT NULL untuk kolom wajib, VARCHAR untuk teks. Dua hal baru untuk diperhatikan:

- username VARCHAR(50) NOT NULL UNIQUE — ingat batasan UNIQUE dari [dokumentasi jobsheet-08 §2.4](#) (dipakai juga untuk no_anggota): **wajib berbeda** untuk setiap baris — masuk akal, karena username dipakai sebagai "identitas login" yang harus unik supaya sistem tahu **akun mana** yang sedang mencoba masuk.
- role VARCHAR(20) NOT NULL DEFAULT 'petugas' — kolom baru untuk menyimpan **peran** pengguna. DEFAULT 'petugas' (ingat konsep DEFAULT dari [dokumentasi jobsheet-08 §2.4](#)) berarti setiap akun baru otomatis berperan sebagai "petugas" kalau tidak ditentukan lain. Kolom ini **disiapkan** untuk kebutuhan masa depan (misalnya membedakan admin dari petugas biasa) — ingat catatan di [README.md](#) jobsheet ini bahwa perbedaan akses berdasarkan role **belum diterapkan**, baru disiapkan kolomnya saja.

2.2 Halaman register.php

```
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
if (isset($_SESSION['user_id'])) {
    header('Location: ../index.php');
    exit;
}
```

```
$page_title = "Registrasi Petugas";  
include __DIR__ . '/../includes/header.php';
```

- `session_status() === PHP_SESSION_NONE` — cara PHP memeriksa apakah session **sudah** dimulai sebelumnya di request ini. Kenapa perlu diperiksa dulu (bukan langsung panggil `session_start()` saja seperti di jobsheet-07/08/09)? Dijelaskan alasannya di [bab 4 §4.4](#).
- `if (isset($_SESSION['user_id'])) { header('Location: ../index.php'); exit; }` — pola baru: kalau pengguna **sudah login**, halaman Registrasi (dan Login, lihat [bab 3](#)) langsung mengalihkannya ke Beranda. Masuk akal — tidak ada gunanya menampilkan form registrasi/login ke seseorang yang sudah punya sesi aktif. Perhatikan target-nya `../index.php`, bukan `/index.php` — karena `register.php` berada satu folder di dalam (`auth/`), path relatif biasa (`../`) sudah cukup untuk sampai ke `index.php` di root proyek, tidak perlu variabel `$base` seperti di `includes/header.php` (ingat [dokumentasi jobsheet-07 §2.3](#) — `$base` dipakai di situ karena `header.php` dipakai bersama oleh halaman di kedalaman berbeda-beda, sedangkan `header('Location: ...')` di sini hanya perlu berlaku untuk file ini sendiri).

Sisa halaman ini adalah form HTML biasa, mengikuti pola `<label> + <input>` yang sudah sangat kamu kenal sejak [dokumentasi jobsheet-01](#) — hanya perhatikan field password memakai `type="password"` (jenis input yang sudah disinggung sejak [dokumentasi jobsheet-04 §2.2](#) sebagai rencana, sekarang benar-benar diimplementasikan) dan atribut `minlength="6"` — validasi HTML baru yang membatasi panjang **minimal** teks yang boleh diketik (beda dari `min/max` pada `type="number"` yang membatasi **nilai** angka, ingat dari [dokumentasi jobsheet-01 §4.4](#)).

2.3 proses_register.php: Validasi & Password Hashing

```
$errors = [];  
if ($nama === '') {  
    $errors[] = "Nama wajib diisi.";  
}  
if ($username === '') {  
    $errors[] = "Username wajib diisi.";  
}  
if (strlen($password) < 6) {  
    $errors[] = "Password minimal 6 karakter.";  
}
```

- Pola validasi ini **identik** dengan `proses_tambah.php` sejak [dokumentasi jobsheet-07 §4.3](#) — array `$errors`, flash message kalau gagal.
- `strlen($password) < 6` — fungsi PHP yang menghitung **panjang** sebuah string. Ini validasi **server-side** untuk aturan yang sama dengan `minlength="6"` di HTML ([§2.2](#)) — ingat prinsip

pentingnya menduplikasi validasi HTML/JS di server dari [dokumentasi jobsheet-07 §4.6](#):
`minlength` bisa dilewati, `strlen()` di server tidak bisa.

```
$cek = $pdo->prepare("SELECT id FROM users WHERE username = :username");
$cek->execute(['username' => $username]);
if ($cek->fetch()) {
    $_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Username sudah digunakan.'];
    header('Location: register.php');
    exit;
}
```

- Ini **pengecekan tambahan** yang belum pernah kamu lihat di `proses_tambah.php` manapun sebelumnya: sebelum benar-benar memasukkan data baru, kode ini **bertanya dulu** ke database "apakah sudah ada pengguna dengan username ini?" Meski kolom `username` sudah diberi batasan `UNIQUE` di database ([§2.1](#)) — yang akan **menolak** `INSERT` yang melanggar — pengecekan manual ini memungkinkan aplikasi memberi **pesan error yang ramah** ("Username sudah digunakan.") alih-alih membiarkan pengguna melihat error database mentah yang membingungkan.

```
$stmt = $pdo->prepare(
    "INSERT INTO users (nama, username, password, role) VALUES (:nama, :username, :password, 'petugas'
);
$stmt->execute([
    'nama' => $nama,
    'username' => $username,
    'password' => password_hash($password, PASSWORD_DEFAULT),
]);
```

- Pola `prepare()` / `execute()` ini sudah kamu kuasai sejak [dokumentasi jobsheet-08 §5](#).
- `password_hash($password, PASSWORD_DEFAULT)` — inilah momen paling penting di seluruh file ini. Password **asli** yang diketik pengguna (`$password`, teks biasa) **tidak pernah** disimpan langsung ke database — yang disimpan adalah **hasil hash-nya**. `PASSWORD_DEFAULT` adalah konstanta PHP yang menunjuk ke algoritma hashing **terbaik yang tersedia saat ini** (PHP bisa memperbarui algoritma defaultnya di versi mendatang tanpa kamu perlu mengubah kode ini) — praktik yang jauh lebih aman dibanding menentukan algoritma spesifik secara manual.
- Perhatikan `'petugas'` di klausa `VALUES` ditulis **langsung** (bukan placeholder `:role`) — karena setiap akun yang mendaftar lewat form ini **selalu** diberi role `'petugas'` tanpa pengecualian, tidak perlu diambil dari input pengguna sama sekali (mencegah seseorang mendaftar sendiri sebagai `'admin'` lewat form publik ini).

Lanjut ke: [Login & Logout](#)

3. Login & Logout

3.1 auth/login.php — Form Login

```
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
if (isset($_SESSION['user_id'])) {
    header('Location: ../index.php');
    exit;
}

$page_title = "Login";
include __DIR__ . '/../includes/header.php';
```

Kerangkanya **identik persis** dengan `register.php` ([dokumentasi jobsheet-10 §2.2](#)) — periksa session, alihkan ke Beranda kalau sudah login, baru tampilkan form (username + password, tanpa `minlength` karena bukan aturan baru yang perlu ditegakkan di sini — panjang password yang benar sudah ditentukan saat Registrasi).

3.2 proses_login.php: Memverifikasi Password

```
$username = trim($_POST['username'] ?? '');
$password = $_POST['password'] ?? '';

$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
$stmt->execute(['username' => $username]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user && password_verify($password, $user['password'])) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['nama'] = $user['nama'];
    $_SESSION['role'] = $user['role'];
    header('Location: ../index.php');
    exit;
}

$_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Username atau password salah.'];
```

```
header('Location: login.php');
exit;
```

- `SELECT * FROM users WHERE username = :username` — mencari pengguna berdasarkan username yang diketik. Ingat pola `WHERE ... = :parameter` dan `->fetch(PDO::FETCH_ASSOC)` (untuk **satu baris**) dari [dokumentasi jobsheet-09 §2.3](#).
- `password_verify($password, $user['password'])` — inilah pasangan dari `password_hash()` yang dibahas di [bab 2 §2.3](#). Fungsi ini menerima **password asli** yang baru saja diketik (`$password`) dan **hash** yang tersimpan di database (`$user['password']`), lalu memeriksa apakah keduanya **cocok** — **tanpa** pernah membalik hash itu kembali menjadi teks asli (memang **tidak mungkin** dibalik, itulah gunanya hashing satu arah dari [dokumentasi jobsheet-10 §1.2](#)). `password_verify()` mengembalikan `true / false`.
- `$user && password_verify(...)` — dua kondisi harus **sama-sama** benar: `$user` harus ditemukan (username-nya memang ada) **dan** password-nya cocok. Kalau username **tidak ditemukan sama sekali**, `$user` bernilai `false`, dan PHP **tidak akan mengevaluasi** `password_verify(...)` sama sekali (perilaku "short-circuit" pada `&&` — kalau bagian kiri sudah `false`, bagian kanan tidak perlu diperiksa lagi) — mencegah error karena mencoba mengakses `$user['password']` padahal `$user` adalah `false`.

Menyimpan Identitas ke Session

```
$_SESSION['user_id'] = $user['id'];
$_SESSION['nama'] = $user['nama'];
$_SESSION['role'] = $user['role'];
```

Setelah kredensial terverifikasi, **3 potong informasi** disimpan ke `$_SESSION` — inilah "bukti" bahwa pengguna ini sudah login, yang akan diperiksa di **setiap** halaman berikutnya oleh `includes/auth.php` ([bab 4](#)) dan ditampilkan di navbar ([bab 5](#)):

Kunci	Dipakai untuk
<code>\$_SESSION['user_id']</code>	Diperiksa <code>auth.php</code> — penanda utama "sudah login"
<code>\$_SESSION['nama']</code>	Ditampilkan di navbar ("halo, nama petugas")
<code>\$_SESSION['role']</code>	Disiapkan untuk kontrol akses berbasis peran di masa depan (ingat catatan di README.md jobsheet ini)

Kenapa Pesan Error-nya Tidak Spesifik?

```
$_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Username atau password salah.'];
```

Perhatikan pesan error ini **sengaja tidak membedakan** apakah username-nya yang salah atau password-nya yang salah — keduanya digabung jadi satu pesan umum "Username atau password salah." Ini **praktik keamanan yang disengaja**: kalau aplikasi menampilkan pesan berbeda ("Username tidak ditemukan" vs "Password salah"), orang yang mencoba menebak-nebak akun orang lain bisa memakai perbedaan pesan itu untuk mengetahui username mana yang **valid** (dengan mencoba banyak username dan melihat pesan mana yang muncul), sebelum mencoba menebak passwordnya. Pesan yang digabung menutup celah informasi ini.

3.3 auth/logout.php: Mengakhiri Sesi

```
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
session_destroy();
header('Location: login.php');
exit;
```

- `session_destroy()` — fungsi PHP bawaan yang **menghapus** seluruh** data session pengguna saat ini sekaligus, termasuk** `user_id`, `nama`, `role` yang tadi disimpan di §3.2. Setelah baris ini, pengguna **tidak lagi dianggap login** — pemeriksaan `isset($_SESSION['user_id'])` di halaman mana pun (termasuk `includes/auth.php`, [bab 4](#)) akan kembali bernilai `false`.
- Setelah sesi dihancurkan, pengguna langsung diarahkan ke halaman Login — alur yang masuk akal setelah logout. Target-nya cukup `login.php` saja (bukan `/auth/login.php` atau `../auth/login.php`) karena `logout.php` **sendiri** sudah berada di dalam folder `auth/` — `login.php` di situ merujuk ke file tetangga di folder yang sama.

3.4 Alur Lengkap: Registrasi → Login → Logout

```
[register.php] --(isi form)--> [proses_register.php]
                --(password_hash, simpan ke DB)--> [redirect ke login.php]

[login.php] --(isi form)--> [proses_login.php]
                --(password_verify cocok)--> [set $_SESSION, redirect ke index.php]
                --(tidak cocok)--> [redirect kembali ke login.php + flash error]
```

```
[halaman mana pun] --(klik Logout)--> [logout.php]
--(session_destroy)--> [redirect ke login.php]
```

Bandungkan alur ini dengan user flow Peminjaman/Pengembalian yang sudah kamu bedah sejak [dokumentasi jobsheet-04 §3](#) — alur "Login" yang saat itu masih berupa **kotak rancangan** kini benar-benar berupa **kode PHP yang berjalan sungguhan**.

Lanjut ke: [Guard Halaman: includes/auth.php](#)

4. Guard Halaman: includes/auth.php

Ini file **terpendek** di jobsheet ini, tapi menyimpan detail paling krusial untuk dipahami — kesalahan urutan sedikit saja bisa membuat seluruh mekanisme proteksinya gagal total.

4.1 Kode Lengkap

```
<?php
// Guard clause: di-include di baris paling atas setiap halaman yang
// membutuhkan login (sebelum header.php mengeluarkan output apa pun),
// agar header('Location: ...') masih bisa dipanggil.
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

if (!isset($_SESSION['user_id'])) {
    header('Location: ../auth/login.php');
    exit;
}
```

includes/auth.php selalu di-require dari halaman yang **satu** folder lebih dalam** dari root proyek (buku/, anggota/, lihat** [§4.7](#)), jadi ../auth/login.php (naik satu folder, lalu turun ke auth/) selalu tepat sasaran — beda dengan includes/header.php yang butuh perhitungan \$base dinamis karena dipakai bersama oleh halaman di kedalaman yang **berbeda-beda** (ingat [dokumentasi jobsheet-07 §2.3](#)).

4.2 Cara Memakainya di Halaman Lain

```
<?php
require __DIR__ . '/../includes/auth.php';
$page_title = "Tambah Buku";
include __DIR__ . '/../includes/header.php';
```

Ini baris **pertama** di buku/tambah.php (ingat dari [dokumentasi jobsheet-10 README](#), halaman ini termasuk yang "terkunci"). Perhatikan `require __DIR__ . '/../includes/auth.php';` dipanggil **sebelum** apa pun yang lain — bahkan sebelum `include header.php`. Urutan ini **bukan kebetulan**, dijelaskan alasannya di [§4.3](#).

4.3 Kenapa Harus Jadi Baris Paling Pertama?

Ingat dari [dokumentasi jobsheet-07 §4.4](#): `header('Location: ...')` **hanya bekerja kalau belum ada output HTML** apa pun** yang terkirim ke browser sebelumnya. Kalau** `include header.php` dipanggil **lebih dulu** — yang langsung mulai mencetak `<!DOCTYPE html>`, `<head>`, dst. — maka saat `auth.php` mencoba memanggil `header('Location: ../auth/login.php')` setelahnya, perintah itu akan **gagal** (biasanya memicu peringatan PHP "headers already sent") karena browser sudah mulai menerima sebagian halaman.

Coba bayangkan urutan yang **salah**:

```
// SALAH — jangan tiru
include __DIR__ . '/../includes/header.php'; // mulai cetak HTML
require __DIR__ . '/../includes/auth.php';    // terlambat! redirect gagal
```

Dengan urutan ini, pengunjung yang belum login akan **tetap melihat** sebagian halaman (header, navbar) sebelum redirect gagal dipanggil — proteksinya bocor. Urutan yang **benar** (`auth.php` dulu, baru `header.php`) memastikan pemeriksaan login selesai **sebelum** satu karakter HTML pun dikirim ke browser.

4.4 Kenapa `session_status()` Diperiksa Dulu?

```
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
```

Bandingkan dengan jobsheet-07/08/09 yang memanggil `session_start()` langsung tanpa pengecekan apa pun. Masalahnya muncul di jobsheet ini karena `session_start()` sekarang dipanggil di **banyak tempat** berbeda** yang berpotensi saling tumpang tindih:**

- `auth.php` memanggilnya (baris pertama di halaman terkunci).
- `header.php` juga memanggilnya (ingat dari [dokumentasi jobsheet-10 §5](#), untuk memeriksa `$sudahLogin`).
- `login.php` / `register.php` / `proses_login.php` / `dst.` juga memanggilnya masing-masing.

Kalau `session_start()` dipanggil **dua kali** dalam satu permintaan halaman yang sama (misalnya sekali oleh `auth.php`, sekali lagi oleh `header.php` yang di-include setelahnya), PHP akan memunculkan **peringatan/error** "session already started." `session_status()` adalah fungsi PHP yang memeriksa **status session saat ini** — `PHP_SESSION_NONE` berarti "belum ada session yang dimulai sama sekali." Dengan membungkus `session_start()` di dalam pengecekan ini, baris tersebut **aman dipanggil berkali-kali** dari file manapun — session hanya benar-benar dimulai **sekali** di panggilan pertama, panggilan-panggilan berikutnya di file lain akan melihat status sudah `PHP_SESSION_ACTIVE` dan melewati `session_start()` begitu saja.

4.5 Inti Pemeriksaan: `isset($_SESSION['user_id'])`

```
if (!isset($_SESSION['user_id'])) {
    header('Location: ../auth/login.php');
    exit;
}
```

Ingat `$_SESSION['user_id']` diset saat Login berhasil ([dokumentasi jobsheet-10 §3.2](#)). Kalau kunci ini **belum ada** (pengunjung belum pernah login, atau sudah logout — ingat `session_destroy()` menghapus semuanya, dari [dokumentasi jobsheet-10 §3.3](#)), `isset(...)` bernilai `true`, dan pengunjung **langsung dialihkan** ke halaman Login — persis pola guard clause `header('Location: ...') + exit;` yang sudah kamu kenal sejak [dokumentasi jobsheet-09 §2.2](#).

4.6 Kenapa Guard Ini Tetap Bekerja Meski Database Belum Tersambung?

Ingat catatan penting di [README.md](#) jobsheet ini:

- "Guard `auth.php` sudah diverifikasi mengembalikan HTTP 302 ke `../auth/login.php` untuk halaman terkunci meski database belum tersambung."* Perhatikan kode `auth.php` di [§4.1](#) **sama sekali tidak menyentuh** `$pdo` atau memanggil `koneksi.php` apa pun — ia **hanya** memeriksa `$_SESSION`, yang sepenuhnya dikelola PHP sendiri tanpa butuh database. Karena `auth.php` dipanggil **paling awal**** ([§4.3](#)), sebelum** baris manapun yang butuh `require koneksi.php`, proteksi ini tetap berfungsi **bahkan kalau PostgreSQL sedang mati total** —

pengunjung yang belum login tetap teralihkan ke Login, tidak pernah sampai ke titik kode yang akan gagal karena database tidak tersedia.

4.7 Halaman Mana Saja yang Memakai Guard Ini?

Ingat dari [README.md](#) jobsheet ini, `auth.php` di-require di:

- `buku/tambah.php`, `buku/edit.php`, `buku/proses_tambah.php`, `buku/proses_edit.php`, `buku/hapus.php`
- **Seluruh** halaman `anggota/*.php`

Sementara `index.php` dan `buku/list.php` **sengaja tidak** memanggil `auth.php` sama sekali — tetap bisa diakses Tamu tanpa login, sesuai pembagian aktor yang sudah dirancang sejak [dokumentasi jobsheet-04](#).

Lanjut ke: [Navbar Dinamis & CSS Pendukung](#)

5. Navbar Dinamis & CSS Pendukung

Bab ini membedah perubahan `includes/header.php` — navbar sekarang "tahu" apakah pengunjungnya sudah login atau belum, dan menyesuaikan tampilannya secara otomatis.

5.1 Kode Lengkap Bagian yang Berubah

```
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
$sudahLogin = isset($_SESSION['user_id']);
?>
...
<header>
    <h1>SIMPUS-Mini</h1>
    <button type="button" id="nav-toggle-btn" class="nav-toggle-label" aria-label="Menu">#9776;
    <nav>
        <ul>
            <li><a href="<?php echo $base; ?>index.php">Beranda</a></li>
            <li><a href="<?php echo $base; ?>buku/list.php">Daftar Buku</a></li>
            <?php if ($sudahLogin): ?>
            <li><a href="<?php echo $base; ?>buku/tambah.php">Tambah Buku</a></li>
            <li><a href="<?php echo $base; ?>anggota/list.php">Daftar Anggota</a></li>
```

```

        <li><a href="<?php echo $base; ?>anggota/tambah.php">Tambah Anggota</a></li>
    <?php endif; ?>
</ul>
</nav>
<div class="auth-status">
    <?php if ($sudahLogin): ?>
        <span><?php echo $_SESSION['nama']; ?></span>
        <a href="<?php echo $base; ?>auth/logout.php">Logout</a>
    <?php else: ?>
        <a href="<?php echo $base; ?>auth/login.php">Login</a>
    <?php endif; ?>
</div>
</header>

```

Perhatikan setiap href diawali <?php echo \$base; ?> (tanpa garis miring / di depan sisa path-nya) — \$base inilah variabel yang dihitung otomatis di baris-baris paling atas header.php, dijelaskan lengkap di [dokumentasi jobsheet-07 §2.3](#). Kalau kamu belum baca bab itu, sebaiknya baca dulu sebelum lanjut — bab ini (navbar dinamis) berfokus ke bagian **login/logout**-nya saja, bukan mengulang penjelasan \$base.

5.2 session_start() yang Diamankan

```

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

```

Ingat dari [dokumentasi jobsheet-10 §4.4](#): header.php **juga** perlu tahu status login (untuk menampilkan \$sudahLogin di bawah), jadi ia juga memanggil session_start() — tapi dengan pengaman session_status() yang sama, supaya tidak bentrok kalau auth.php sudah memanggilnya lebih dulu di halaman yang terkunci ([dokumentasi jobsheet-10 §4.2](#)).

5.3 Satu Variabel, Dipakai Berkali-Kali: \$sudahLogin

```

$sudahLogin = isset($_SESSION['user_id']);

```

Nilai true / false ini disimpan **satu kali** ke variabel \$sudahLogin, lalu dipakai **berulang** di bawahnya — daripada menulis isset(\$_SESSION['user_id']) berkali-kali di setiap tempat yang membutuhkannya. Kebiasaan menyimpan hasil pengecekan ke variabel seperti ini membuat kode lebih ringkas dan lebih mudah dibaca.

5.4 Menu yang Muncul-Hilang: `<?php if ($sudahLogin): ?>`

```
<li><a href="<?php echo $base; ?>index.php">Beranda</a></li>
<li><a href="<?php echo $base; ?>buku/list.php">Daftar Buku</a></li>
<?php if ($sudahLogin): ?>
<li><a href="<?php echo $base; ?>buku/tambah.php">Tambah Buku</a></li>
<li><a href="<?php echo $base; ?>anggota/list.php">Daftar Anggota</a></li>
<li><a href="<?php echo $base; ?>anggota/tambah.php">Tambah Anggota</a></li>
<?php endif; ?>
```

Ingat sintaks alternatif `if (...): ... endif;` dari [dokumentasi jobsheet-07 §2.5](#). Menu "Beranda" dan "Daftar Buku" **selalu tampil** (sesuai dengan `index.php / buku/list.php` yang tetap publik, ingat dari [dokumentasi jobsheet-10 README](#)). Tiga menu lainnya (Tambah Buku, Daftar Anggota, Tambah Anggota) hanya muncul kalau `$sudahLogin` bernilai `true` — **konsisten** dengan halaman mana saja yang sebenarnya dikunci oleh `auth.php` ([dokumentasi jobsheet-10 §4.7](#)).

Catatan penting: menyembunyikan menu ini di navbar **bukan** mekanisme keamanan itu sendiri — ini murni **kenyamanan tampilan** (tidak menampilkan tautan ke halaman yang toh akan menolak akses Tamu). Proteksi sesungguhnya tetap `includes/auth.php` ([bab 4](#)) — bahkan kalau seseorang mengetik `/buku/tambah.php` **langsung** di address bar tanpa mengklik menu apa pun, `auth.php` tetap akan mengalihkannya ke Login.

5.5 Status Login di Pojok Kanan: `.auth-status`

```
<div class="auth-status">
  <?php if ($sudahLogin): ?>
    <span><?php echo $_SESSION['nama']; ?></span>
    <a href="<?php echo $base; ?>auth/logout.php">Logout</a>
  <?php else: ?>
    <a href="<?php echo $base; ?>auth/login.php">Login</a>
  <?php endif; ?>
</div>
```

- Kalau **sudah login**: menampilkan nama petugas (`$_SESSION['nama']`), ingat disimpan saat Login di [dokumentasi jobsheet-10 §3.2](#)) dan tautan Logout.
- Kalau **belum login**: menampilkan tautan Login saja.

Ini **persis** mewujudkan wireframe Dashboard yang sudah kamu bedah sejak [dokumentasi jobsheet-04 §2.3](#):

- "(Nama Petugas) Logout"* — potongan wireframe yang saat itu masih berupa teks ASCII kini benar-benar ditampilkan dari data session sungguhan.

5.6 CSS Pendukung: `.auth-status`

```
.auth-status {  
  display: flex;  
  align-items: center;  
  gap: 0.75rem;  
  color: #fff;  
}  
  
.auth-status a {  
  color: #fff;  
  text-decoration: underline;  
}
```

- `display: flex; align-items: center; gap: 0.75rem;` — ingat Flexbox dari [dokumentasi jobsheet-02 §4](#): menyusun nama petugas dan tautan Logout (atau tautan Login saja) sejajar horizontal dengan jarak rapi, sejajar secara vertikal dengan elemen lain di header.
- `color: #fff` — teks putih, konsisten dengan warna teks navbar lain di header biru tema ([dokumentasi jobsheet-02 §4.6](#)).
- `.auth-status a { text-decoration: underline; }` — tautan Login/ Logout di sini sengaja **diberi** garis bawah (berbeda dari tautan menu navbar utama yang tidak bergaris bawah, ingat dari [dokumentasi jobsheet-02 §4.6](#)) — memberi sedikit pembeda visual bahwa area ini adalah "status akun", terpisah secara fungsi dari menu navigasi biasa.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

6. Rangkuman & Latihan Lanjutan

6.1 Rangkuman Keseluruhan Jobsheet 10

Bagian	Konsep yang Dipelajari
Konsep Dasar Autentikasi	Autentikasi vs otorisasi, kenapa password harus di-hash

Bagian	Konsep yang Dipelajari
Tabel <code>users</code> & Registrasi	Skema <code>users</code> , <code>password_hash()</code> , cek username duplikat
Login & Logout	<code>password_verify()</code> , menyimpan identitas ke <code>\$_SESSION</code> , <code>session_destroy()</code>
Guard <code>auth.php</code>	Urutan <code>include</code> yang kritis, <code>session_status()</code> , guard yang bekerja tanpa database
Navbar Dinamis & CSS	Menu & status login kondisional, <code>\$sudahLogin</code>

6.2 Konsep Inti yang Perlu Diingat

1. **Password harus di-hash, tidak pernah disimpan apa adanya** — `password_hash()` saat menyimpan, `password_verify()` saat memeriksa, keduanya **tidak pernah** membutuhkan/menghasilkan password asli dalam bentuk yang bisa dibaca kembali ([bab 1 §1.2](#)).
2. **Urutan `include` / `require` bisa jadi krusial** — `auth.php` wajib dipanggil sebelum ada output HTML apa pun, karena `header('Location: ...')` gagal kalau dipanggil terlambat ([bab 4 §4.3](#)).
3. `session_status()` **mencegah konflik** `session_start()` **ganda**, penting begitu banyak file berbeda perlu memeriksa/memulai session yang sama ([bab 4 §4.4](#)).
4. **Guard clause otorisasi seharusnya tidak bergantung pada database**** — supaya proteksi tetap berfungsi meski ada bagian lain** dari aplikasi yang gagal ([bab 4 §4.6](#)).
5. **Menyembunyikan menu di UI bukan pengganti otorisasi sungguhan**** — keduanya perlu ada bersamaan: UI yang rapi **dan**** guard yang benar-benar memblokir akses langsung ([bab 5 §5.4](#)).

6.3 Cara Mencoba Sendiri

1. Jalankan skema tambahan (ingat dari [README.md](#)):

```
psql -d simpus_mini -f sql/02_users.sql
```

2. Jalankan `php -S localhost:8000` (atau lewat Laragon), buka `http://localhost:8000/buku/tambah.php` **langsung** tanpa login — amati kamu diarahkan ke halaman Login (bukti guard `auth.php` bekerja).

3. Klik "Daftar di sini", isi form Registrasi — amati password yang kamu ketik **tidak pernah muncul** di database secara langsung (kalau penasaran, lihat isi kolom `password` di tabel `users` lewat `psql`, akan terlihat teks acakan panjang, bukan password aslimu).
4. Login dengan akun yang baru dibuat — amati navbar berubah: muncul menu Tambah Buku/Daftar Anggota/Tambah Anggota, dan nama petugas + tautan Logout di pojok kanan.
5. Buka kembali `/buku/tambah.php` — sekarang **berhasil** diakses (dibandingkan langkah 2 tadi).
6. Klik Logout — amati navbar kembali seperti semula (menu terbatas, tautan Login muncul lagi), dan coba akses `/buku/tambah.php` lagi — kembali diarahkan ke Login.
7. Coba **salah** memasukkan password saat Login — amati pesan error umum "Username atau password salah." muncul (ingat alasannya dari [bab 3 §3.2](#)).

6.4 Ide Latihan Tambahan (Opsional)

1. **Terapkan kontrol akses berbasis role** — sesuai catatan di [README.md](#) jobsheet ini yang menyebutnya sebagai tugas mandiri: buat aturan misalnya hanya `role === 'admin'` yang boleh mengakses `anggota/hapus.php`, sementara `'petugas'` biasa hanya boleh melihat dan menambah data. Petunjuk: kamu perlu menambah pengecekan baru **setelah** `require auth.php`, memeriksa `$_SESSION['role']`.
2. **Tambah "Ingat Saya" (Remember Me)** — cari tahu lewat dokumentasi PHP resmi bagaimana cookie dengan masa berlaku panjang bisa dipakai untuk menjaga sesi login tetap aktif meski browser ditutup (petunjuk: fungsi `setcookie()`), lalu diskusikan sendiri risiko keamanannya dibanding sekadar mengandalkan `$_SESSION` biasa.
3. **Batasi percobaan Login yang gagal** — tambahkan penghitung percobaan gagal per username (bisa disimpan sementara di `$_SESSION` untuk latihan), dan tampilkan peringatan setelah beberapa kali gagal berturut-turut — langkah awal mencegah serangan
 - brute-force* menebak password.
4. **Uji coba mematikan PostgreSQL** sesuai catatan di [README.md](#) jobsheet ini — coba hentikan sementara layanan PostgreSQL di komputermu, lalu akses `/buku/tambah.php` tanpa login — buktikan sendiri kamu tetap diarahkan ke Login (bukan melihat error koneksi database), sesuai penjelasan di [bab 4 §4.6](#).

Kalau ada bagian yang masih membingungkan, terutama soal urutan `include/require` di [bab 4](#), coba baca ulang sambil membayangkan skenario jika urutannya dibalik — memahami **kenapa** sesuatu harus dilakukan dalam urutan tertentu jauh lebih berharga daripada sekadar menghafal urutannya.